

O'REILLY®

Second
Edition

Practical Statistics for Data Scientists

50+ Essential Concepts Using R and Python



Peter Bruce, Andrew Bruce
& Peter Gedeck

Practical Statistics for Data Scientists

Statistical methods are a key part of data science, yet few data scientists have formal statistical training. Courses and books on basic statistics rarely cover the topic from a data science perspective. The second edition of this popular guide adds comprehensive examples in Python, provides practical guidance on applying statistical methods to data science, tells you how to avoid their misuse, and gives you advice on what's important and what's not.

Many data science resources incorporate statistical methods but lack a deeper statistical perspective. If you're familiar with the R or Python programming languages and have some exposure to statistics, this quick reference bridges the gap in an accessible, readable format.

With this book, you'll learn:

- Why exploratory data analysis is a key preliminary step in data science
- How random sampling can reduce bias and yield a higher-quality dataset, even with big data
- How the principles of experimental design yield definitive answers to questions
- How to use regression to estimate outcomes and detect anomalies
- Key classification techniques for predicting which categories a record belongs to
- Statistical machine learning methods that "learn" from data
- Unsupervised learning methods for extracting meaning from unlabeled data

"This book is not another statistics textbook nor a machine learning manual. It's much better: it makes the connection between useful statistical terms and principles and today's data mining lingo and practices, with clear explanations and plenty of examples. This is a terrific reference for data science beginners and old timers."

—Galit Shmueli

lead author of the best-selling series
Data Mining for Business Analytics and
Distinguished Professor, National Tsing
Hua University, Taiwan

Peter Bruce is the founder of the Institute for Statistics Education at Statistics.com.

Andrew Bruce is a principal research scientist at Amazon and has over 30 years of experience in statistics and data science.

Peter Gedeck is a senior data scientist at Collaborative Drug Discovery, developing machine learning algorithms to predict properties of drug candidates.

DATA SCIENCE | STATISTICS

US \$69.99

CAN \$92.99

ISBN: 978-1-492-07294-2



Twitter: @oreillymedia
facebook.com/oreilly

SECOND EDITION

Practical Statistics for Data Scientists

50+ Essential Concepts Using R and Python

Peter Bruce, Andrew Bruce, and Peter Gedeck

Beijing • Boston • Farnham • Sebastopol • Tokyo

O'REILLY®

Practical Statistics for Data Scientists

by Peter Bruce, Andrew Bruce, and Peter Gedeck

Copyright © 2020 Peter Bruce, Andrew Bruce, and Peter Gedeck. All rights reserved.

Printed in the United States of America.

Published by O'Reilly Media, Inc., 1005 Gravenstein Highway North, Sebastopol, CA 95472.

O'Reilly books may be purchased for educational, business, or sales promotional use. Online editions are also available for most titles (<http://oreilly.com>). For more information, contact our corporate/institutional sales department: 800-998-9938 or corporate@oreilly.com.

Editor: Nicole Tache

Production Editor: Kristen Brown

Copyeditor: Piper Editorial

Proofreader: Arthur Johnson

Indexer: Ellen Troutman-Zaig

Interior Designer: David Futato

Cover Designer: Karen Montgomery

Illustrator: Rebecca Demarest

May 2017: First Edition

May 2020: Second Edition

Revision History for the Second Edition

2020-04-10: First Release

2020-06-19: Second Release

2020-10-02: Third Release

See <http://oreilly.com/catalog/errata.csp?isbn=9781492072942> for release details.

The O'Reilly logo is a registered trademark of O'Reilly Media, Inc. *Practical Statistics for Data Scientists*, the cover image, and related trade dress are trademarks of O'Reilly Media, Inc.

The views expressed in this work are those of the authors, and do not represent the publisher's views. While the publisher and the authors have used good faith efforts to ensure that the information and instructions contained in this work are accurate, the publisher and the authors disclaim all responsibility for errors or omissions, including without limitation responsibility for damages resulting from the use of or reliance on this work. Use of the information and instructions contained in this work is at your own risk. If any code samples or other technology this work contains or describes is subject to open source licenses or the intellectual property rights of others, it is your responsibility to ensure that your use thereof complies with such licenses and/or rights.

978-1-492-07294-2

[LSCH]

Peter Bruce and Andrew Bruce would like to dedicate this book to the memories of our parents, Victor G. Bruce and Nancy C. Bruce, who cultivated a passion for math and science; and to our early mentors John W. Tukey and Julian Simon and our lifelong friend Geoff Watson, who helped inspire us to pursue a career in statistics.

Peter Gedeck would like to dedicate this book to Tim Clark and Christian Kramer, with deep thanks for their scientific collaboration and friendship.

Table of Contents

Preface.....	xiii
1. Exploratory Data Analysis.....	1
Elements of Structured Data	2
Further Reading	4
Rectangular Data	4
Data Frames and Indexes	6
Nonrectangular Data Structures	6
Further Reading	7
Estimates of Location	7
Mean	9
Median and Robust Estimates	10
Example: Location Estimates of Population and Murder Rates	12
Further Reading	13
Estimates of Variability	13
Standard Deviation and Related Estimates	14
Estimates Based on Percentiles	16
Example: Variability Estimates of State Population	18
Further Reading	19
Exploring the Data Distribution	19
Percentiles and Boxplots	20
Frequency Tables and Histograms	22
Density Plots and Estimates	24
Further Reading	26
Exploring Binary and Categorical Data	27
Mode	29
Expected Value	29
Probability	30

Further Reading	30
Correlation	30
Scatterplots	34
Further Reading	36
Exploring Two or More Variables	36
Hexagonal Binning and Contours (Plotting Numeric Versus Numeric Data)	36
Two Categorical Variables	39
Categorical and Numeric Data	41
Visualizing Multiple Variables	43
Further Reading	46
Summary	46
2. Data and Sampling Distributions.....	47
Random Sampling and Sample Bias	48
Bias	50
Random Selection	51
Size Versus Quality: When Does Size Matter?	52
Sample Mean Versus Population Mean	53
Further Reading	53
Selection Bias	54
Regression to the Mean	55
Further Reading	57
Sampling Distribution of a Statistic	57
Central Limit Theorem	60
Standard Error	60
Further Reading	61
The Bootstrap	61
Resampling Versus Bootstrapping	65
Further Reading	65
Confidence Intervals	65
Further Reading	68
Normal Distribution	69
Standard Normal and QQ-Plots	71
Long-Tailed Distributions	73
Further Reading	75
Student's t-Distribution	75
Further Reading	78
Binomial Distribution	78
Further Reading	80
Chi-Square Distribution	80
Further Reading	81
F-Distribution	82

Further Reading	82
Poisson and Related Distributions	82
Poisson Distributions	83
Exponential Distribution	84
Estimating the Failure Rate	84
Weibull Distribution	85
Further Reading	86
Summary	86
3. Statistical Experiments and Significance Testing.....	87
A/B Testing	88
Why Have a Control Group?	90
Why Just A/B? Why Not C, D,...?	91
Further Reading	92
Hypothesis Tests	93
The Null Hypothesis	94
Alternative Hypothesis	95
One-Way Versus Two-Way Hypothesis Tests	95
Further Reading	96
Resampling	96
Permutation Test	97
Example: Web Stickiness	98
Exhaustive and Bootstrap Permutation Tests	102
Permutation Tests: The Bottom Line for Data Science	102
Further Reading	103
Statistical Significance and p-Values	103
p-Value	106
Alpha	107
Type 1 and Type 2 Errors	109
Data Science and p-Values	109
Further Reading	110
t-Tests	110
Further Reading	112
Multiple Testing	112
Further Reading	116
Degrees of Freedom	116
Further Reading	118
ANOVA	118
F-Statistic	121
Two-Way ANOVA	123
Further Reading	124
Chi-Square Test	124

Chi-Square Test: A Resampling Approach	124
Chi-Square Test: Statistical Theory	127
Fisher's Exact Test	128
Relevance for Data Science	130
Further Reading	131
Multi-Arm Bandit Algorithm	131
Further Reading	134
Power and Sample Size	135
Sample Size	136
Further Reading	138
Summary	139
4. Regression and Prediction.....	141
Simple Linear Regression	141
The Regression Equation	143
Fitted Values and Residuals	146
Least Squares	148
Prediction Versus Explanation (Profiling)	149
Further Reading	150
Multiple Linear Regression	150
Example: King County Housing Data	151
Assessing the Model	153
Cross-Validation	155
Model Selection and Stepwise Regression	156
Weighted Regression	159
Further Reading	161
Prediction Using Regression	161
The Dangers of Extrapolation	161
Confidence and Prediction Intervals	161
Factor Variables in Regression	163
Dummy Variables Representation	164
Factor Variables with Many Levels	167
Ordered Factor Variables	169
Interpreting the Regression Equation	169
Correlated Predictors	170
Multicollinearity	172
Confounding Variables	172
Interactions and Main Effects	174
Regression Diagnostics	176
Outliers	177
Influential Values	179
Heteroskedasticity, Non-Normality, and Correlated Errors	182

Partial Residual Plots and Nonlinearity	185
Polynomial and Spline Regression	187
Polynomial	188
Splines	189
Generalized Additive Models	192
Further Reading	193
Summary	194
5. Classification.....	195
Naive Bayes	196
Why Exact Bayesian Classification Is Impractical	197
The Naive Solution	198
Numeric Predictor Variables	200
Further Reading	201
Discriminant Analysis	201
Covariance Matrix	202
Fisher's Linear Discriminant	203
A Simple Example	204
Further Reading	207
Logistic Regression	208
Logistic Response Function and Logit	208
Logistic Regression and the GLM	210
Generalized Linear Models	212
Predicted Values from Logistic Regression	212
Interpreting the Coefficients and Odds Ratios	213
Linear and Logistic Regression: Similarities and Differences	214
Assessing the Model	216
Further Reading	219
Evaluating Classification Models	219
Confusion Matrix	221
The Rare Class Problem	223
Precision, Recall, and Specificity	223
ROC Curve	224
AUC	226
Lift	228
Further Reading	229
Strategies for Imbalanced Data	230
Undersampling	231
Oversampling and Up/Down Weighting	232
Data Generation	233
Cost-Based Classification	234
Exploring the Predictions	234

Further Reading	236
Summary	236
6. Statistical Machine Learning.....	237
K-Nearest Neighbors	238
A Small Example: Predicting Loan Default	239
Distance Metrics	241
One Hot Encoder	242
Standardization (Normalization, z-Scores)	243
Choosing K	246
KNN as a Feature Engine	247
Tree Models	249
A Simple Example	250
The Recursive Partitioning Algorithm	252
Measuring Homogeneity or Impurity	254
Stopping the Tree from Growing	256
Predicting a Continuous Value	257
How Trees Are Used	258
Further Reading	259
Bagging and the Random Forest	259
Bagging	260
Random Forest	261
Variable Importance	265
Hyperparameters	269
Boosting	270
The Boosting Algorithm	271
XGBoost	272
Regularization: Avoiding Overfitting	274
Hyperparameters and Cross-Validation	279
Summary	282
7. Unsupervised Learning.....	283
Principal Components Analysis	284
A Simple Example	285
Computing the Principal Components	288
Interpreting Principal Components	289
Correspondence Analysis	292
Further Reading	294
K-Means Clustering	294
A Simple Example	295
K-Means Algorithm	298
Interpreting the Clusters	299

Selecting the Number of Clusters	302
Hierarchical Clustering	304
A Simple Example	305
The Dendrogram	306
The Agglomerative Algorithm	308
Measures of Dissimilarity	309
Model-Based Clustering	311
Multivariate Normal Distribution	311
Mixtures of Normals	312
Selecting the Number of Clusters	315
Further Reading	318
Scaling and Categorical Variables	318
Scaling the Variables	319
Dominant Variables	321
Categorical Data and Gower's Distance	322
Problems with Clustering Mixed Data	325
Summary	326
Bibliography.....	327
Index.....	329

Preface

This book is aimed at the data scientist with some familiarity with the *R* and/or *Python* programming languages, and with some prior (perhaps spotty or ephemeral) exposure to statistics. Two of the authors came to the world of data science from the world of statistics, and have some appreciation of the contribution that statistics can make to the art of data science. At the same time, we are well aware of the limitations of traditional statistics instruction: statistics as a discipline is a century and a half old, and most statistics textbooks and courses are laden with the momentum and inertia of an ocean liner. All the methods in this book have some connection—historical or methodological—to the discipline of statistics. Methods that evolved mainly out of computer science, such as neural nets, are not included.

Two goals underlie this book:

- To lay out, in digestible, navigable, and easily referenced form, key concepts from statistics that are relevant to data science.
- To explain which concepts are important and useful from a data science perspective, which are less so, and why.

Conventions Used in This Book

The following typographical conventions are used in this book:

Italic

Indicates new terms, URLs, email addresses, filenames, and file extensions.

Constant width

Used for program listings, as well as within paragraphs to refer to program elements such as variable or function names, databases, data types, environment variables, statements, and keywords.

Constant width bold

Shows commands or other text that should be typed literally by the user.

Key Terms

Data science is a fusion of multiple disciplines, including statistics, computer science, information technology, and domain-specific fields. As a result, several different terms could be used to reference a given concept. Key terms and their synonyms will be highlighted throughout the book in a sidebar such as this.



This element signifies a tip or suggestion.



This element signifies a general note.



This element indicates a warning or caution.

Using Code Examples

In all cases, this book gives code examples first in *R* and then in *Python*. In order to avoid unnecessary repetition, we generally show only output and plots created by the *R* code. We also skip the code required to load the required packages and data sets. You can find the complete code as well as the data sets for download at <https://github.com/gedeck/practical-statistics-for-data-scientists>.

This book is here to help you get your job done. In general, if example code is offered with this book, you may use it in your programs and documentation. You do not need to contact us for permission unless you're reproducing a significant portion of the code. For example, writing a program that uses several chunks of code from this book does not require permission. Selling or distributing examples from O'Reilly books does require permission. Answering a question by citing this book and quoting example code does not require permission. Incorporating a significant amount of

example code from this book into your product's documentation does require permission.

We appreciate, but do not require, attribution. An attribution usually includes the title, author, publisher, and ISBN. For example: “*Practical Statistics for Data Scientists* by Peter Bruce, Andrew Bruce, and Peter Gedeck (O’Reilly). Copyright 2020 Peter Bruce, Andrew Bruce, and Peter Gedeck, 978-1-492-07294-2.”

If you feel your use of code examples falls outside fair use or the permission given above, feel free to contact us at permissions@oreilly.com.

O’Reilly Online Learning



For more than 40 years, *O’Reilly Media* has provided technology and business training, knowledge, and insight to help companies succeed.

Our unique network of experts and innovators share their knowledge and expertise through books, articles, and our online learning platform. O’Reilly’s online learning platform gives you on-demand access to live training courses, in-depth learning paths, interactive coding environments, and a vast collection of text and video from O’Reilly and 200+ other publishers. For more information, visit <http://oreilly.com>.

How to Contact Us

Please address comments and questions concerning this book to the publisher:

O’Reilly Media, Inc.
1005 Gravenstein Highway North
Sebastopol, CA 95472
800-998-9938 (in the United States or Canada)
707-829-0515 (international or local)
707-829-0104 (fax)

We have a web page for this book, where we list errata, examples, and any additional information. You can access this page at https://oreil.ly/practicalStats_dataSci_2e.

Email bookquestions@oreilly.com to comment or ask technical questions about this book.

For news and more information about our books and courses, see our website at <http://oreilly.com>.

Find us on Facebook: <http://facebook.com/oreilly>

Follow us on Twitter: <http://twitter.com/oreillymedia>

Watch us on YouTube: <http://www.youtube.com/oreillymedia>

Acknowledgments

The authors acknowledge the many people who helped make this book a reality.

Gerhard Pilcher, CEO of the data mining firm Elder Research, saw early drafts of the book and gave us detailed and helpful corrections and comments. Likewise, Anya McGuirk and Wei Xiao, statisticians at SAS, and Jay Hilfiger, fellow O'Reilly author, provided helpful feedback on initial drafts of the book. Toshiaki Kurokawa, who translated the first edition into Japanese, did a comprehensive job of reviewing and correcting in the process. Aaron Schumacher and Walter Paczkowski thoroughly reviewed the second edition of the book and provided numerous helpful and valuable suggestions for which we are extremely grateful. Needless to say, any errors that remain are ours alone.

At O'Reilly, Shannon Cutt has shepherded us through the publication process with good cheer and the right amount of prodding, while Kristen Brown smoothly took our book through the production phase. Rachel Monaghan and Eliahu Sussman corrected and improved our writing with care and patience, while Ellen Troutman-Zaig prepared the index. Nicole Tache took over the reins for the second edition and has both guided the process effectively and provided many good editorial suggestions to improve the readability of the book for a broad audience. We also thank Marie Beaugureau, who initiated our project at O'Reilly, as well as Ben Bengfort, O'Reilly author and Statistics.com instructor, who introduced us to O'Reilly.

We, and this book, have also benefited from the many conversations Peter has had over the years with Galit Shmueli, coauthor on other book projects.

Finally, we would like to especially thank Elizabeth Bruce and Deborah Donnell, whose patience and support made this endeavor possible.

Exploratory Data Analysis

This chapter focuses on the first step in any data science project: exploring the data.

Classical statistics focused almost exclusively on *inference*, a sometimes complex set of procedures for drawing conclusions about large populations based on small samples. In 1962, John W. Tukey (Figure 1-1) called for a reformation of statistics in his seminal paper “The Future of Data Analysis” [Tukey-1962]. He proposed a new scientific discipline called *data analysis* that included statistical inference as just one component. Tukey forged links to the engineering and computer science communities (he coined the terms *bit*, short for binary digit, and *software*), and his original tenets are surprisingly durable and form part of the foundation for data science. The field of exploratory data analysis was established with Tukey’s 1977 now-classic book *Exploratory Data Analysis* [Tukey-1977]. Tukey presented simple plots (e.g., boxplots, scatterplots) that, along with summary statistics (mean, median, quantiles, etc.), help paint a picture of a data set.

With the ready availability of computing power and expressive data analysis software, exploratory data analysis has evolved well beyond its original scope. Key drivers of this discipline have been the rapid development of new technology, access to more and bigger data, and the greater use of quantitative analysis in a variety of disciplines. David Donoho, professor of statistics at Stanford University and former undergraduate student of Tukey’s, authored an excellent article based on his presentation at the Tukey Centennial workshop in Princeton, New Jersey [Donoho-2015]. Donoho traces the genesis of data science back to Tukey’s pioneering work in data analysis.



Figure 1-1. John Tukey, the eminent statistician whose ideas developed over 50 years ago form the foundation of data science

Elements of Structured Data

Data comes from many sources: sensor measurements, events, text, images, and videos. The *Internet of Things* (IoT) is spewing out streams of information. Much of this data is unstructured: images are a collection of pixels, with each pixel containing RGB (red, green, blue) color information. Texts are sequences of words and nonword characters, often organized by sections, subsections, and so on. Clickstreams are sequences of actions by a user interacting with an app or a web page. In fact, a major challenge of data science is to harness this torrent of raw data into actionable information. To apply the statistical concepts covered in this book, unstructured raw data must be processed and manipulated into a structured form. One of the commonest forms of structured data is a table with rows and columns—as data might emerge from a relational database or be collected for a study.

There are two basic types of structured data: numeric and categorical. Numeric data comes in two forms: *continuous*, such as wind speed or time duration, and *discrete*, such as the count of the occurrence of an event. *Categorical* data takes only a fixed set of values, such as a type of TV screen (plasma, LCD, LED, etc.) or a state name (Alabama, Alaska, etc.). *Binary* data is an important special case of categorical data that takes on only one of two values, such as 0/1, yes/no, or true/false. Another useful type of categorical data is *ordinal* data in which the categories are ordered; an example of this is a numerical rating (1, 2, 3, 4, or 5).

Why do we bother with a taxonomy of data types? It turns out that for the purposes of data analysis and predictive modeling, the data type is important to help determine the type of visual display, data analysis, or statistical model. In fact, data science software, such as *R* and *Python*, uses these data types to improve computational performance. More important, the data type for a variable determines how software will handle computations for that variable.

Key Terms for Data Types

Numeric

Data that are expressed on a numeric scale.

Continuous

Data that can take on any value in an interval. (*Synonyms*: interval, float, numeric)

Discrete

Data that can take on only integer values, such as counts. (*Synonyms*: integer, count)

Categorical

Data that can take on only a specific set of values representing a set of possible categories. (*Synonyms*: enums, enumerated, factors, nominal)

Binary

A special case of categorical data with just two categories of values, e.g., 0/1, true/false. (*Synonyms*: dichotomous, logical, indicator, boolean)

Ordinal

Categorical data that has an explicit ordering. (*Synonym*: ordered factor)

Software engineers and database programmers may wonder why we even need the notion of *categorical* and *ordinal* data for analytics. After all, categories are merely a collection of text (or numeric) values, and the underlying database automatically handles the internal representation. However, explicit identification of data as categorical, as distinct from text, does offer some advantages:

- Knowing that data is categorical can act as a signal telling software how statistical procedures, such as producing a chart or fitting a model, should behave. In particular, ordinal data can be represented as an `ordered.factor` in *R*, preserving a user-specified ordering in charts, tables, and models. In *Python*, `scikit-learn` supports ordinal data with the `sklearn.preprocessing.OrdinalEncoder`.
- Storage and indexing can be optimized (as in a relational database).
- The possible values a given categorical variable can take are enforced in the software (like an enum).

The third “benefit” can lead to unintended or unexpected behavior: the default behavior of data import functions in *R* (e.g., `read.csv`) is to automatically convert a text column into a `factor`. Subsequent operations on that column will assume that the only allowable values for that column are the ones originally imported, and assigning a new text value will introduce a warning and produce an NA (missing

value). The `pandas` package in *Python* will not make such a conversion automatically. However, you can specify a column as categorical explicitly in the `read_csv` function.

Key Ideas

- Data is typically classified in software by type.
- Data types include numeric (continuous, discrete) and categorical (binary, ordinal).
- Data typing in software acts as a signal to the software on how to process the data.

Further Reading

- The [pandas documentation](#) describes the different data types and how they can be manipulated in *Python*.
- Data types can be confusing, since types may overlap, and the taxonomy in one software may differ from that in another. The [R Tutorial website](#) covers the taxonomy for *R*.
- Databases are more detailed in their classification of data types, incorporating considerations of precision levels, fixed- or variable-length fields, and more; see the [W3Schools guide to SQL](#).

Rectangular Data

The typical frame of reference for an analysis in data science is a *rectangular data* object, like a spreadsheet or database table.

Rectangular data is the general term for a two-dimensional matrix with rows indicating records (cases) and columns indicating features (variables); *data frame* is the specific format in *R* and *Python*. The data doesn't always start in this form: unstructured data (e.g., text) must be processed and manipulated so that it can be represented as a set of features in the rectangular data (see “[Elements of Structured Data](#)” on page 2). Data in relational databases must be extracted and put into a single table for most data analysis and modeling tasks.

Key Terms for Rectangular Data

Data frame

Rectangular data (like a spreadsheet) is the basic data structure for statistical and machine learning models.

Feature

A column within a table is commonly referred to as a *feature*.

Synonyms

attribute, input, predictor, variable

Outcome

Many data science projects involve predicting an *outcome*—often a yes/no outcome (in [Table 1-1](#), it is “auction was competitive or not”). The *features* are sometimes used to predict the *outcome* in an experiment or a study.

Synonyms

dependent variable, response, target, output

Records

A row within a table is commonly referred to as a *record*.

Synonyms

case, example, instance, observation, pattern, sample

Table 1-1. A typical data frame format

Category	currency	sellerRating	Duration	endDay	ClosePrice	OpenPrice	Competitive?
Music/Movie/Game	US	3249	5	Mon	0.01	0.01	0
Music/Movie/Game	US	3249	5	Mon	0.01	0.01	0
Automotive	US	3115	7	Tue	0.01	0.01	0
Automotive	US	3115	7	Tue	0.01	0.01	0
Automotive	US	3115	7	Tue	0.01	0.01	0
Automotive	US	3115	7	Tue	0.01	0.01	0
Automotive	US	3115	7	Tue	0.01	0.01	1
Automotive	US	3115	7	Tue	0.01	0.01	1

In [Table 1-1](#), there is a mix of measured or counted data (e.g., duration and price) and categorical data (e.g., category and currency). As mentioned earlier, a special form of categorical variable is a binary (yes/no or 0/1) variable, seen in the rightmost column in [Table 1-1](#)—an indicator variable showing whether an auction was competitive (had multiple bidders) or not. This indicator variable also happens to be an *outcome* variable, when the scenario is to predict whether an auction is competitive or not.

Data Frames and Indexes

Traditional database tables have one or more columns designated as an index, essentially a row number. This can vastly improve the efficiency of certain database queries. In *Python*, with the `pandas` library, the basic rectangular data structure is a `DataFrame` object. By default, an automatic integer index is created for a `DataFrame` based on the order of the rows. In `pandas`, it is also possible to set multilevel/hierarchical indexes to improve the efficiency of certain operations.

In *R*, the basic rectangular data structure is a `data.frame` object. A `data.frame` also has an implicit integer index based on the row order. The native *R* `data.frame` does not support user-specified or multilevel indexes, though a custom key can be created through the `row.names` attribute. To overcome this deficiency, two new packages are gaining widespread use: `data.table` and `dplyr`. Both support multilevel indexes and offer significant speedups in working with a `data.frame`.



Terminology Differences

Terminology for rectangular data can be confusing. Statisticians and data scientists use different terms for the same thing. For a statistician, *predictor variables* are used in a model to predict a *response* or *dependent variable*. For a data scientist, *features* are used to predict a *target*. One synonym is particularly confusing: computer scientists will use the term *sample* for a single row; a *sample* to a statistician means a collection of rows.

Nonrectangular Data Structures

There are other data structures besides rectangular data.

Time series data records successive measurements of the same variable. It is the raw material for statistical forecasting methods, and it is also a key component of the data produced by devices—the Internet of Things.

Spatial data structures, which are used in mapping and location analytics, are more complex and varied than rectangular data structures. In the *object* representation, the focus of the data is an object (e.g., a house) and its spatial coordinates. The *field* view, by contrast, focuses on small units of space and the value of a relevant metric (pixel brightness, for example).

Graph (or network) data structures are used to represent physical, social, and abstract relationships. For example, a graph of a social network, such as Facebook or LinkedIn, may represent connections between people on the network. Distribution hubs connected by roads are an example of a physical network. Graph structures are useful for certain types of problems, such as network optimization and recommender systems.

Each of these data types has its specialized methodology in data science. The focus of this book is on rectangular data, the fundamental building block of predictive modeling.



Graphs in Statistics

In computer science and information technology, the term *graph* typically refers to a depiction of the connections among entities, and to the underlying data structure. In statistics, *graph* is used to refer to a variety of plots and *visualizations*, not just of connections among entities, and the term applies only to the visualization, not to the data structure.

Key Ideas

- The basic data structure in data science is a rectangular matrix in which rows are records and columns are variables (features).
- Terminology can be confusing; there are a variety of synonyms arising from the different disciplines that contribute to data science (statistics, computer science, and information technology).

Further Reading

- [Documentation on data frames in R](#)
- [Documentation on data frames in Python](#)

Estimates of Location

Variables with measured or count data might have thousands of distinct values. A basic step in exploring your data is getting a “typical value” for each feature (variable): an estimate of where most of the data is located (i.e., its central tendency).

Key Terms for Estimates of Location

Mean

The sum of all values divided by the number of values.

Synonym
average

Weighted mean

The sum of all values times a weight divided by the sum of the weights.

Synonym
weighted average

Median

The value such that one-half of the data lies above and below.

Synonym
50th percentile

Percentile

The value such that P percent of the data lies below.

Synonym
quantile

Weighted median

The value such that one-half of the sum of the weights lies above and below the sorted data.

Trimmed mean

The average of all values after dropping a fixed number of extreme values.

Synonym
truncated mean

Robust

Not sensitive to extreme values.

Synonym
resistant

Outlier

A data value that is very different from most of the data.

Synonym
extreme value

At first glance, summarizing data might seem fairly trivial: just take the *mean* of the data. In fact, while the mean is easy to compute and expedient to use, it may not always be the best measure for a central value. For this reason, statisticians have developed and promoted several alternative estimates to the mean.



Metrics and Estimates

Statisticians often use the term *estimate* for a value calculated from the data at hand, to draw a distinction between what we see from the data and the theoretical true or exact state of affairs. Data scientists and business analysts are more likely to refer to such a value as a *metric*. The difference reflects the approach of statistics versus that of data science: accounting for uncertainty lies at the heart of the discipline of statistics, whereas concrete business or organizational objectives are the focus of data science. Hence, statisticians estimate, and data scientists measure.

Mean

The most basic estimate of location is the mean, or *average* value. The mean is the sum of all values divided by the number of values. Consider the following set of numbers: {3 5 1 2}. The mean is $(3 + 5 + 1 + 2) / 4 = 11 / 4 = 2.75$. You will encounter the symbol $\bar{x}$ (pronounced “x-bar”) being used to represent the mean of a sample from a population. The formula to compute the mean for a set of n values $x_1, x_2, \dots, x_n$ is:

$$\text{Mean} = \bar{x} = \frac{\sum_{i=1}^n x_i}{n}$$



N (or n) refers to the total number of records or observations. In statistics it is capitalized if it is referring to a population, and lower-case if it refers to a sample from a population. In data science, that distinction is not vital, so you may see it both ways.

A variation of the mean is a *trimmed mean*, which you calculate by dropping a fixed number of sorted values at each end and then taking an average of the remaining values. Representing the sorted values by $x_{(1)}, x_{(2)}, \dots, x_{(n)}$ where $x_{(1)}$ is the smallest value and $x_{(n)}$ the largest, the formula to compute the trimmed mean with p smallest and largest values omitted is:

$$\text{Trimmed mean} = \bar{x} = \frac{\sum_{i=p+1}^{n-p} x_{(i)}}{n - 2p}$$

A trimmed mean eliminates the influence of extreme values. For example, in international diving the top score and bottom score from five judges are dropped, and **the final score is the average of the scores from the three remaining judges**. This makes it difficult for a single judge to manipulate the score, perhaps to favor their country's contestant. Trimmed means are widely used, and in many cases are preferable to using the ordinary mean—see **“Median and Robust Estimates” on page 10** for further discussion.

Another type of mean is a *weighted mean*, which you calculate by multiplying each data value x_i by a user-specified weight w_i and dividing their sum by the sum of the weights. The formula for a weighted mean is:

$$\text{Weighted mean} = \bar{x}_w = \frac{\sum_{i=1}^n w_i x_i}{\sum_{i=1}^n w_i}$$

There are two main motivations for using a weighted mean:

- Some values are intrinsically more variable than others, and highly variable observations are given a lower weight. For example, if we are taking the average from multiple sensors and one of the sensors is less accurate, then we might downweight the data from that sensor.
- The data collected does not equally represent the different groups that we are interested in measuring. For example, because of the way an online experiment was conducted, we may not have a set of data that accurately reflects all groups in the user base. To correct that, we can give a higher weight to the values from the groups that were underrepresented.

Median and Robust Estimates

The *median* is the middle number on a sorted list of the data. If there is an even number of data values, the middle value is one that is not actually in the data set, but rather the average of the two values that divide the sorted data into upper and lower halves. Compared to the mean, which uses all observations, the median depends only on the values in the center of the sorted data. While this might seem to be a disadvantage, since the mean is much more sensitive to the data, there are many instances in which the median is a better metric for location. Let's say we want to look at typical household incomes in neighborhoods around Lake Washington in Seattle. In comparing the Medina neighborhood to the Windermere neighborhood, using the mean would produce very different results because Bill Gates lives in Medina. If we use the median, it won't matter how rich Bill Gates is—the position of the middle observation will remain the same.

For the same reasons that one uses a weighted mean, it is also possible to compute a *weighted median*. As with the median, we first sort the data, although each data value has an associated weight. Instead of the middle number, the weighted median is a value such that the sum of the weights is equal for the lower and upper halves of the sorted list. Like the median, the weighted median is robust to outliers.

Outliers

The median is referred to as a *robust* estimate of location since it is not influenced by *outliers* (extreme cases) that could skew the results. An outlier is any value that is very distant from the other values in a data set. The exact definition of an outlier is somewhat subjective, although certain conventions are used in various data summaries and plots (see “[Percentiles and Boxplots](#)” on page 20). Being an outlier in itself does not make a data value invalid or erroneous (as in the previous example with Bill Gates). Still, outliers are often the result of data errors such as mixing data of different units (kilometers versus meters) or bad readings from a sensor. When outliers are the result of bad data, the mean will result in a poor estimate of location, while the median will still be valid. In any case, outliers should be identified and are usually worthy of further investigation.



Anomaly Detection

In contrast to typical data analysis, where outliers are sometimes informative and sometimes a nuisance, in *anomaly detection* the points of interest are the outliers, and the greater mass of data serves primarily to define the “normal” against which anomalies are measured.

The median is not the only robust estimate of location. In fact, a trimmed mean is widely used to avoid the influence of outliers. For example, trimming the bottom and top 10% (a common choice) of the data will provide protection against outliers in all but the smallest data sets. The trimmed mean can be thought of as a compromise between the median and the mean: it is robust to extreme values in the data, but uses more data to calculate the estimate for location.



Other Robust Metrics for Location

Statisticians have developed a plethora of other estimators for location, primarily with the goal of developing an estimator more robust than the mean and also more efficient (i.e., better able to discern small location differences between data sets). While these methods are potentially useful for small data sets, they are not likely to provide added benefit for large or even moderately sized data sets.

Example: Location Estimates of Population and Murder Rates

Table 1-2 shows the first few rows in the data set containing population and murder rates (in units of murders per 100,000 people per year) for each US state (2010 Census).

Table 1-2. A few rows of the data.frame state of population and murder rate by state

	State	Population	Murder rate	Abbreviation
1	Alabama	4,779,736	5.7	AL
2	Alaska	710,231	5.6	AK
3	Arizona	6,392,017	4.7	AZ
4	Arkansas	2,915,918	5.6	AR
5	California	37,253,956	4.4	CA
6	Colorado	5,029,196	2.8	CO
7	Connecticut	3,574,097	2.4	CT
8	Delaware	897,934	5.8	DE

Compute the mean, trimmed mean, and median for the population using R:

```
> state <- read.csv('state.csv')
> mean(state[['Population']])
[1] 6162876
> mean(state[['Population']], trim=0.1)
[1] 4783697
> median(state[['Population']])
[1] 4436370
```

To compute mean and median in *Python* we can use the pandas methods of the data frame. The trimmed mean requires the `trim_mean` function in `scipy.stats`:

```
state = pd.read_csv('state.csv')
state['Population'].mean()
trim_mean(state['Population'], 0.1)
state['Population'].median()
```

The mean is bigger than the trimmed mean, which is bigger than the median.

This is because the trimmed mean excludes the largest and smallest five states (`trim=0.1` drops 10% from each end). If we want to compute the average murder rate for the country, we need to use a weighted mean or median to account for different populations in the states. Since base R doesn't have a function for weighted median, we need to install a package such as `matrixStats`:

```
> weighted.mean(state[['Murder.Rate']], w=state[['Population']])
[1] 4.445834
> library('matrixStats')
```

```
> weightedMedian(state[['Murder.Rate']], w=state[['Population']])  
[1] 4.4
```

Weighted mean is available with NumPy. For weighted median, we can use the specialized package **wquantiles**:

```
np.average(state['Murder.Rate'], weights=state['Population'])  
wquantiles.median(state['Murder.Rate'], weights=state['Population'])
```

In this case, the weighted mean and the weighted median are about the same.

Key Ideas

- The basic metric for location is the mean, but it can be sensitive to extreme values (outlier).
- Other metrics (median, trimmed mean) are less sensitive to outliers and unusual distributions and hence are more robust.

Further Reading

- The Wikipedia article on **central tendency** contains an extensive discussion of various measures of location.
- John Tukey's 1977 classic *Exploratory Data Analysis* (Pearson) is still widely read.

Estimates of Variability

Location is just one dimension in summarizing a feature. A second dimension, *variability*, also referred to as *dispersion*, measures whether the data values are tightly clustered or spread out. At the heart of statistics lies variability: measuring it, reducing it, distinguishing random from real variability, identifying the various sources of real variability, and making decisions in the presence of it.

Key Terms for Variability Metrics

Deviations

The difference between the observed values and the estimate of location.

Synonyms

errors, residuals

Variance

The sum of squared deviations from the mean divided by $n - 1$ where n is the number of data values.

Synonym
mean-squared-error

Standard deviation

The square root of the variance.

Mean absolute deviation

The mean of the absolute values of the deviations from the mean.

Synonyms
l1-norm, Manhattan norm

Median absolute deviation from the median

The median of the absolute values of the deviations from the median.

Range

The difference between the largest and the smallest value in a data set.

Order statistics

Metrics based on the data values sorted from smallest to biggest.

Synonym
ranks

Percentile

The value such that P percent of the values take on this value or less and $(100-P)$ percent take on this value or more.

Synonym
quantile

Interquartile range

The difference between the 75th percentile and the 25th percentile.

Synonym
IQR

Just as there are different ways to measure location (mean, median, etc.), there are also different ways to measure variability.

Standard Deviation and Related Estimates

The most widely used estimates of variation are based on the differences, or *deviations*, between the estimate of location and the observed data. For a set of data $\{1, 4, 4\}$, the mean is 3 and the median is 4. The deviations from the mean are the differences: $1 - 3 = -2$, $4 - 3 = 1$, $4 - 3 = 1$. These deviations tell us how dispersed the data is around the central value.

One way to measure variability is to estimate a typical value for these deviations. Averaging the deviations themselves would not tell us much—the negative deviations offset the positive ones. In fact, the sum of the deviations from the mean is precisely zero. Instead, a simple approach is to take the average of the absolute values of the deviations from the mean. In the preceding example, the absolute value of the deviations is {2 1 1}, and their average is $(2 + 1 + 1) / 3 = 1.33$. This is known as the *mean absolute deviation* and is computed with the formula:

$$\text{Mean absolute deviation} = \frac{\sum_{i=1}^n |x_i - \bar{x}|}{n}$$

where $\bar{x}$ is the sample mean.

The best-known estimates of variability are the *variance* and the *standard deviation*, which are based on squared deviations. The variance is an average of the squared deviations, and the standard deviation is the square root of the variance:

$$\begin{aligned} \text{Variance} &= s^2 = \frac{\sum_{i=1}^n (x_i - \bar{x})^2}{n - 1} \\ \text{Standard deviation} &= s = \sqrt{\text{Variance}} \end{aligned}$$

The standard deviation is much easier to interpret than the variance since it is on the same scale as the original data. Still, with its more complicated and less intuitive formula, it might seem peculiar that the standard deviation is preferred in statistics over the mean absolute deviation. It owes its preeminence to statistical theory: mathematically, working with squared values is much more convenient than absolute values, especially for statistical models.

Degrees of Freedom, and n or $n - 1$?

In statistics books, there is always some discussion of why we have $n - 1$ in the denominator in the variance formula, instead of n , leading into the concept of *degrees of freedom*. This distinction is not important since n is generally large enough that it won't make much difference whether you divide by n or $n - 1$. But in case you are interested, here is the story. It is based on the premise that you want to make estimates about a population, based on a sample.

If you use the intuitive denominator of n in the variance formula, you will underestimate the true value of the variance and the standard deviation in the population. This is referred to as a *biased* estimate. However, if you divide by $n - 1$ instead of n , the variance becomes an *unbiased* estimate.

To fully explain why using n leads to a biased estimate involves the notion of degrees of freedom, which takes into account the number of constraints in computing an estimate. In this case, there are $n - 1$ degrees of freedom since there is one constraint: the standard deviation depends on calculating the sample mean. For most problems, data scientists do not need to worry about degrees of freedom.

Neither the variance, the standard deviation, nor the mean absolute deviation is robust to outliers and extreme values (see “Median and Robust Estimates” on page 10 for a discussion of robust estimates for location). The variance and standard deviation are especially sensitive to outliers since they are based on the squared deviations.

A robust estimate of variability is the *median absolute deviation from the median* or MAD:

$$\text{Median absolute deviation} = \text{Median}(|x_1 - m|, |x_2 - m|, \dots, |x_N - m|)$$

where m is the median. Like the median, the MAD is not influenced by extreme values. It is also possible to compute a trimmed standard deviation analogous to the trimmed mean (see “Mean” on page 9).



The variance, the standard deviation, the mean absolute deviation, and the median absolute deviation from the median are not equivalent estimates, even in the case where the data comes from a normal distribution. In fact, the standard deviation is always greater than the mean absolute deviation, which itself is greater than the median absolute deviation. Sometimes, the median absolute deviation is multiplied by a constant scaling factor to put the MAD on the same scale as the standard deviation in the case of a normal distribution. The commonly used factor of 1.4826 means that 50% of the normal distribution fall within the range $\pm \text{MAD}$ (see, e.g., <https://oreil.ly/SfDk2>).

Estimates Based on Percentiles

A different approach to estimating dispersion is based on looking at the spread of the sorted data. Statistics based on sorted (ranked) data are referred to as *order statistics*. The most basic measure is the *range*: the difference between the largest and smallest numbers. The minimum and maximum values themselves are useful to know and are helpful in identifying outliers, but the range is extremely sensitive to outliers and not very useful as a general measure of dispersion in the data.

To avoid the sensitivity to outliers, we can look at the range of the data after dropping values from each end. Formally, these types of estimates are based on differences

between *percentiles*. In a data set, the P th percentile is a value such that at least P percent of the values take on this value or less and at least $(100 - P)$ percent of the values take on this value or more. For example, to find the 80th percentile, sort the data. Then, starting with the smallest value, proceed 80 percent of the way to the largest value. Note that the median is the same thing as the 50th percentile. The percentile is essentially the same as a *quantile*, with quantiles indexed by fractions (so the .8 quantile is the same as the 80th percentile).

A common measurement of variability is the difference between the 25th percentile and the 75th percentile, called the *interquartile range* (or IQR). Here is a simple example: {3,1,5,3,6,7,2,9}. We sort these to get {1,2,3,3,5,6,7,9}. The 25th percentile is at 2.5, and the 75th percentile is at 6.5, so the interquartile range is $6.5 - 2.5 = 4$. Software can have slightly differing approaches that yield different answers (see the following tip); typically, these differences are smaller.

For very large data sets, calculating exact percentiles can be computationally very expensive since it requires sorting all the data values. Machine learning and statistical software use special algorithms, such as [Zhang-Wang-2007], to get an approximate percentile that can be calculated very quickly and is guaranteed to have a certain accuracy.



Percentile: Precise Definition

If we have an even number of data (n is even), then the percentile is ambiguous under the preceding definition. In fact, we could take on any value between the order statistics $x_{(j)}$ and $x_{(j+1)}$ where j satisfies:

$$100 * \frac{j}{n} \leq P < 100 * \frac{j+1}{n}$$

Formally, the percentile is the weighted average:

$$\text{Percentile}(P) = (1 - w)x_{(j)} + wx_{(j+1)}$$

for some weight w between 0 and 1. Statistical software has slightly differing approaches to choosing w . In fact, the *R* function `quantile` offers nine different alternatives to compute the quantile. Except for small data sets, you don't usually need to worry about the precise way a percentile is calculated. *Python*'s `numpy.quantile` supports five approaches, with linear interpolation being the default.

Example: Variability Estimates of State Population

Table 1-3 (repeated from **Table 1-2** for convenience) shows the first few rows in the data set containing population and murder rates for each state.

Table 1-3. A few rows of the data.frame state of population and murder rate by state

	State	Population	Murder rate	Abbreviation
1	Alabama	4,779,736	5.7	AL
2	Alaska	710,231	5.6	AK
3	Arizona	6,392,017	4.7	AZ
4	Arkansas	2,915,918	5.6	AR
5	California	37,253,956	4.4	CA
6	Colorado	5,029,196	2.8	CO
7	Connecticut	3,574,097	2.4	CT
8	Delaware	897,934	5.8	DE

Using R's built-in functions for the standard deviation, the interquartile range (IQR), and the median absolute deviation from the median (MAD), we can compute estimates of variability for the state population data:

```
> sd(state[['Population']])  
[1] 6848235  
> IQR(state[['Population']])  
[1] 4847308  
> mad(state[['Population']])  
[1] 3849870
```

The `pandas` data frame provides methods for calculating standard deviation and quantiles. Using the quantiles, we can easily determine the IQR. For the robust MAD, we use the function `robust.scale.mad` from the `statsmodels` package:

```
state['Population'].std()  
state['Population'].quantile(0.75) - state['Population'].quantile(0.25)  
robust.scale.mad(state['Population'])
```

The standard deviation is almost twice as large as the MAD (in R, by default, the scale of the MAD is adjusted to be on the same scale as the mean). This is not surprising since the standard deviation is sensitive to outliers.

Key Ideas

- Variance and standard deviation are the most widespread and routinely reported statistics of variability.
- Both are sensitive to outliers.
- More robust metrics include mean absolute deviation, median absolute deviation from the median, and percentiles (quantiles).

Further Reading

- David Lane’s online statistics resource has a [section on percentiles](#).
- Kevin Davenport has a [useful post on R-Bloggers](#) about deviations from the median and their robust properties.

Exploring the Data Distribution

Each of the estimates we’ve covered sums up the data in a single number to describe the location or variability of the data. It is also useful to explore how the data is distributed overall.

Key Terms for Exploring the Distribution

Boxplot

A plot introduced by Tukey as a quick way to visualize the distribution of data.

Synonym

box and whiskers plot

Frequency table

A tally of the count of numeric data values that fall into a set of intervals (bins).

Histogram

A plot of the frequency table with the bins on the x-axis and the count (or proportion) on the y-axis. While visually similar, bar charts should not be confused with histograms. See “[Exploring Binary and Categorical Data](#)” on page 27 for a discussion of the difference.

Density plot

A smoothed version of the histogram, often based on a *kernel density estimate*.